

Linked List in Python – Complete Implementation Guide

Highlighted lines = Pointer Update (core logic)
Min pointer updates = minimum next/prev changes

1. Singly Linked List

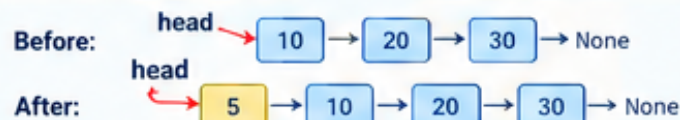
1.1 Insertion Operations (Singly Linked List)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```



1. Insert at Beginning

Min Pointer Updates: 1

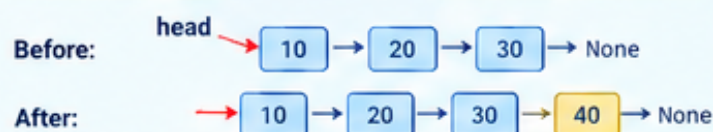


```
def insert_at_beginning(self, data):
    new_node = Node(data)
    new_node.next = self.head # pointer update 1
    self.head = new_node
    return
```

Time: O(1)
Space: O(1)
Pointer Updates: 1
(2 if counting head)

2. Insert at End

Min Pointer Updates: 1

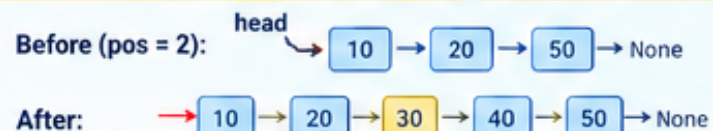


```
def insert_at_end(self, data):
    new_node = Node(data)
    if not self.head:
        self.head = new_node
        return
    temp = self.head
    while temp.next:
        temp = temp.next
    temp.next = new_node # pointer update 1
```

Time: O(n)
Space: O(1)
Pointer Updates: 1

3. Insert at Given Position (0-indexed)

Min Pointer Updates: 2



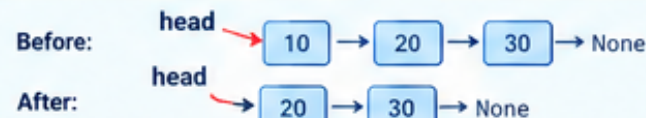
```
def insert_at_position(self, data, pos):
    if pos == 0:
        self.insert_at_beginning(data)
        return
    new_node = Node(data)
    temp = self.head
    for _ in range(pos-1):
        if not temp: return
        temp = temp.next
    new_node.next = temp.next # pointer update 1
    temp.next = new_node # pointer update 2
```

Time: O(n)
Space: O(1)
Pointer Updates: 2

1.2 Deletion Operations (Singly Linked List)

1. Delete from Beginning

Min Pointer Updates: 1



```
def delete_from_beginning(self):
    if not self.head: return
    temp = self.head
    self.head = self.head.next # pointer update 1
    del temp
```

Time: O(1)
Space: O(1)
Pointer Updates: 1

2. Delete from End

Min Pointer Updates: 1



```
def delete_from_end(self):
    if not self.head: return
    if not self.head.next:
        del self.head
        self.head = None
        return
    temp = self.head
    while temp.next.next:
        temp = temp.next
    del temp.next
    temp.next = None # pointer update 1
```

Time: O(n)
Space: O(1)
Pointer Updates: 1

3. Delete from Given Position (0-indexed)

Min Pointer Updates: 1



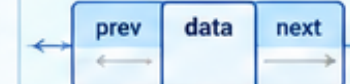
```
def delete_from_position(self, pos):
    if not self.head: return
    if pos == 0:
        temp = self.head
        self.head = self.head.next # pointer update 1
        del temp
        return
    temp = self.head
    for _ in range(pos-1):
        if not temp or not temp.next: return
        temp = temp.next
    del_node = temp.next
    temp.next = del_node.next # pointer update 1
    del del_node
```

Time: O(n)
Space: O(1)
Pointer Updates: 2

2. Doubly Linked List

```
class DNode:
```

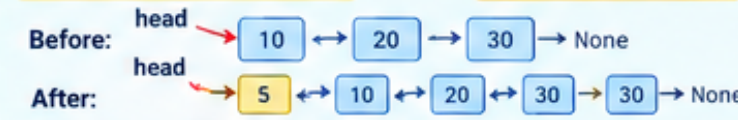
```
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```



2.1 Insertion Operations (Doubly Linked List)

1. Insert at Beginning

Min Pointer Updates: 2



```
def insert_at_beginning(self, data):
    new_node = DNode(data)
    new_node.next = self.head # update 1
    if self.head:
        self.head.prev = new_node # update 2
    self.head = new_node
```

Time: O(1)
Space: O(1)
Pointer Updates: 2

2. Insert at End

Min Pointer Updates: 2

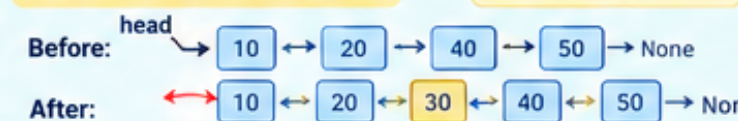


```
def insert_at_end(self, data):
    new_node = DNode(data)
    if not self.head:
        self.head = new_node
        return
    temp = self.head
    while temp.next:
        temp = temp.next
    temp.next = new_node # update 1
    new_node.prev = temp # update 2
```

Time: O(n)
Space: O(1)
Pointer Updates: 2

3. Insert at Given Position

Min Pointer Updates: 4



```
def insert_at_position(self, data, pos):
    if pos == 0:
        self.insert_at_beginning(data)
        return
    new_node = DNode(data)
    temp = self.head
    for _ in range(pos-1):
        if not temp: return
        temp = temp.next
    new_node.next = temp.next # update 1
    new_node.prev = temp # update 2
    if temp.next:
        temp.next.prev = new_node # update 3
    temp.next = new_node # update 4
```

Time: O(n)
Space: O(1)
Pointer Updates: 4

2.2 Deletion Operations (Doubly Linked List)

1. Delete from Beginning

Min Pointer Updates: 2

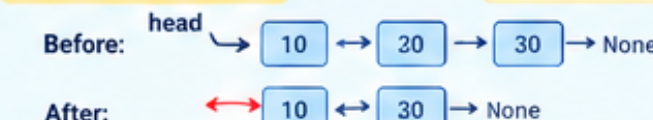


```
def delete_from_beginning(self):
    if not self.head: return
    temp = self.head
    self.head = self.head.next # update 1
    if self.head:
        self.head.prev = None # update 2
    del temp
```

Time: O(1)
Space: O(1)
Pointer Updates: 2

2. Delete from End

Min Pointer Updates: 2



```
def delete_from_end(self):
    if not self.head: return
    if not self.head.next:
        del self.head
        self.head = None
        return
    temp = self.head
    while temp.next:
        temp = temp.next
    temp.prev.next = None # update 1
    del temp # node removed (update 2)
```

Time: O(n)
Space: O(1)
Pointer Updates: 2

3. Delete from Given Position

Min Pointer Updates: 4



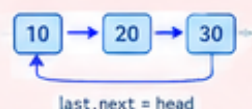
```
def delete_from_position(self, pos):
    if not self.head: return
    if pos == 0:
        self.delete_from_beginning(); return
    temp = self.head
    for _ in range(pos):
        if not temp: return
        temp = temp.next
    if not temp: return
    if temp.prev:
        temp.prev.next = temp.next # update 1
    if temp.next:
        temp.next.prev = temp.prev # update 2
    del temp
```

Time: O(n)
Space: O(1)
Pointer Updates: 2
(4 with explicit handling)

3. Circular Linked List (Singly Circular and Doubly Circular)

3.1 Singly Circular Linked List

```
class CNode:
    def __init__(self, data):
        self.data = data
        self.next = None
```



Operation	Key Logic	Min Pointer Updates (excl. temp)	Time	Space
Insert at beginning	new.next = head, last.next = new, head = new	2 (3 with head)	O(1)	O(1)
Insert at end	new.next = head, last.next = new	1	O(n)	O(1)
Insert at given position	new.next = temp.next, temp.next = new	2	O(n)	O(1)
Delete from beginning	last.next = head.next, head = head.next	2	O(n)	O(1)
Delete from end	find second last, temp.next = head	1	O(n)	O(1)
Delete from given position	temp.next = temp.next.next	1	O(n)	O(1)

3.2 Doubly Circular Linked List

```
class CDNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```

Operation	Key Logic	Min Pointer Updates	Time	Space
Insert at beginning	new.next = head, new.prev = last, last.next = new, head.prev = new	4	O(1)	O(1)
Insert at end	new.prev = last, new.next = head, last.next = new, head.prev = new	4	O(1)	O(1)
Insert at given position	Update next/prev of 4 nodes	4	O(n)	O(1)
Delete from beginning	last.next = head.next, head.next.prev = last, head = head.next, head.prev = last	4	O(1)	O(1)
Delete from end	last = last.prev, last.next = head, head.prev = last	3	O(1)	O(1)
Delete from given position	Update next/prev of 4 nodes	4	O(n)	O(1)